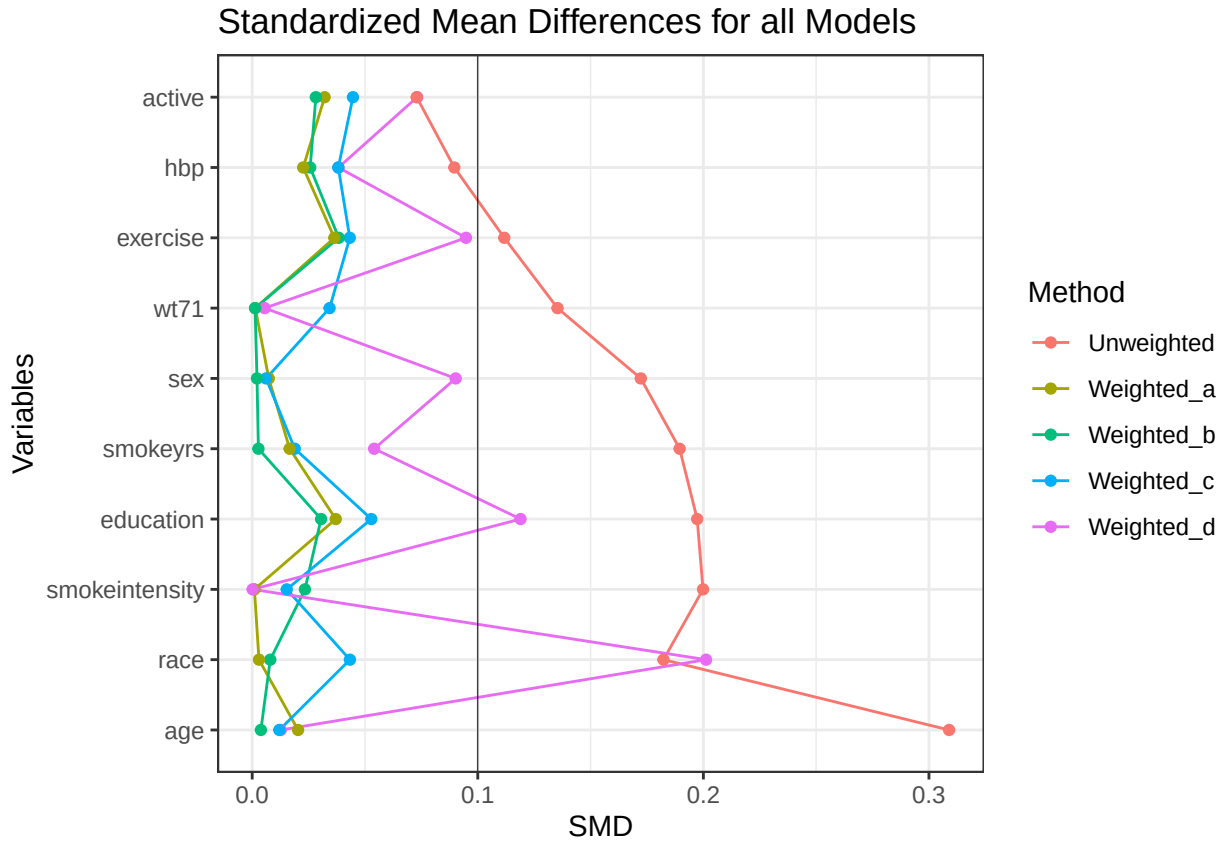
PubH7485 Homework3 Key

2024-10-05

Question 1 (30 points total, 20 points for the plot, 10 points for the choice of model and justification)

From the plot, we can see that the propensity score weighting is useful in achieving balance. Generally, Model b with the nonlinear terms seems to work the best, with all SMDs less than 0.1 and having a smallest average SMD of 0.0164.

(The key is using random forest to select the set of covariates for Model d, however, any optimal method is acceptable.)



Model	Average_SMD
Model A	0.0177772
Model B	0.0164195
Model C	0.0308860
Model D	0.0688957

Question 2 (30 points total, 10 points for each outcome)

The calculated results are summarized in the following tables:

Table 2: Results for Weight Change

	ATE	Standard Error	95% CI
PSS	3.310	0.466	2.397, 4.222
IPW1	3.330	0.477	2.395, 4.264
IPW2	3.345	0.476	2.411, 4.278

Table 3: Results for SBP

	ATE	Standard Error	95% CI
PSS	1.140	0.992	-0.803, 3.084
IPW1	0.772	1.726	-2.610, 4.155
IPW2	1.120	0.980	-0.800, 3.040

Table 4: Results for DBP

	ATE	Standard Error	95% CI
PSS	1.219	0.624	-0.005, 2.442
IPW1	1.056	1.137	-1.174, 3.285
IPW2	1.267	0.615	0.063, 2.472

Note that depend on different models you are using, the results may be a little different.

Appendix - R code

Question 1

```r
### Logistic regression with main effects
ps_model_a <- glm(qsmk ~ sex + race + age + education + smokeintensity + smokeyrs + exercise + active +
nhefs$ps_a <- predict(ps_model_a, type = "response")
nhefs$weight_a <- with(nhefs, ifelse(qsmk == "Yes", 1 / ps_a, 1 / (1 - ps_a)))

# Calculate weighted and unweighted SMDs
dataSvy_a <- svydesign(ids = ~ 1, data = nhefs, weights = ~ weight_a)
tabUnweighted_a <- CreateTableOne(vars = adj_vars, strata = "qsmk", data = nhefs, test = FALSE)
tabWeighted_a <- svyCreateTableOne(vars = adj_vars, strata = "qsmk", data = dataSvy_a, test = FALSE)

dataPlot_a <- data.frame(variable  = rownames(ExtractSmd(tabUnweighted_a)),
                         Unweighted = as.numeric(ExtractSmd(tabUnweighted_a)),
                         Weighted_a  = as.numeric(ExtractSmd(tabWeighted_a)))

# Convert to long format for ggplot
```

```r
dataPlotMelt_a <- melt(data = dataPlot_a,
                       id.vars = c("variable"),
                       variable.name = "Method",
                       value.name = "SMD")

### Logistic regression with main effects and nonlinear terms in the continuous covariates
ps_model_b <- glm(qsmk ~ sex + race + poly(age, 2) + education + poly(smokeintensity, 2) + poly(smokeyrs
nhefs$ps_b <- predict(ps_model_b, type = "response")
nhefs$weight_b <- with(nhefs, ifelse(qsmk == "Yes", 1 / ps_b, 1 / (1 - ps_b)))

dataSvy_b <- svydesign(ids = ~ 1, data = nhefs, weights = ~ weight_b)
tabUnweighted_b <- CreateTableOne(vars = adj_vars, strata = "qsmk", data = nhefs, test = FALSE)
tabWeighted_b <- svyCreateTableOne(vars = adj_vars, strata = "qsmk", data = dataSvy_b, test = FALSE)

dataPlot_b <- data.frame(variable  = rownames(ExtractSmd(tabUnweighted_b)),
                         Unweighted = as.numeric(ExtractSmd(tabUnweighted_b)),
                         Weighted_b  = as.numeric(ExtractSmd(tabWeighted_b)))

dataPlotMelt_b <- melt(data = dataPlot_b,
                       id.vars = c("variable"),
                       variable.name = "Method",
                       value.name = "SMD")

### Allowing for all pairwise interactions and performing some form of variable selection (e.g., forwar
full_model_c <- glm(qsmk ~ (sex + race + age + education + smokeintensity + smokeyrs + exercise + activ
ps_model_c <- step(full_model_c, trace=0, direction="forward", criteria="AIC")
nhefs$ps_c <- predict(ps_model_c, type="response")
nhefs$weight_c <- with(nhefs, ifelse(qsmk == "Yes", 1 / ps_c, 1 / (1 - ps_c)))

dataSvy_c <- svydesign(ids = ~ 1, data = nhefs, weights = ~ weight_c)
tabUnweighted_c <- CreateTableOne(vars = adj_vars, strata = "qsmk", data = nhefs, test = FALSE)
tabWeighted_c <- svyCreateTableOne(vars = adj_vars, strata = "qsmk", data = dataSvy_c, test = FALSE)

dataPlot_c <- data.frame(variable  = rownames(ExtractSmd(tabUnweighted_c)),
                         Unweighted = as.numeric(ExtractSmd(tabUnweighted_c)),
                         Weighted_c = as.numeric(ExtractSmd(tabWeighted_c)))

dataPlotMelt_c <- melt(data = dataPlot_c,
                       id.vars = c("variable"),
                       variable.name = "Method",
                       value.name = "SMD")


### A logistic regression model considering all available baseline covariates and fit a random forest m
rf_model <- randomForest(as.formula(paste("qsmk ~", paste(all_covars[-c(1:3,13)], collapse = " + "))), 
important_vars <- importance(rf_model)[order(importance(rf_model)[, "MeanDecreaseGini"], decreasing = TR
top_10_vars <- row.names(as.data.frame(important_vars[1:10]))

formula_d <- as.formula(paste("qsmk ~", paste(top_10_vars, collapse=" + ")))

ps_model_d <- glm(formula_d, family = binomial(), data = nhefs)
nhefs$ps_d <- predict(ps_model_d, type = "response")
nhefs$weight_d <- with(nhefs, ifelse(qsmk == "Yes", 1 / ps_d, 1 / (1 - ps_d)))
```

```r
dataSvy_d <- svydesign(ids = ~ 1, data = nhefs, weights = ~ weight_d)
tabUnweighted_d <- CreateTableOne(vars = adj_vars, strata = "qsmk", data = nhefs, test = FALSE)
tabWeighted_d <- svyCreateTableOne(vars = adj_vars, strata = "qsmk", data = dataSvy_d, test = FALSE)

dataPlot_d <- data.frame(variable   = rownames(ExtractSmd(tabUnweighted_d)),
                         Unweighted = as.numeric(ExtractSmd(tabUnweighted_d)),
                         Weighted_d  = as.numeric(ExtractSmd(tabWeighted_d)))

dataPlotMelt_d <- melt(data = dataPlot_d,
                       id.vars = c("variable"),
                       variable.name = "Method",
                       value.name = "SMD")

# Prepare dataset for ggplot
dataPlotMelt <- rbind(dataPlotMelt_a, dataPlotMelt_b[11:20, ], dataPlotMelt_c[11:20, ], dataPlotMelt_d[
dataPlotMelt %>%
  arrange(desc(SMD), variable) %>%
  mutate(variable_ordered = factor(variable, levels = unique(variable))) %>%
  ggplot(aes(x = variable_ordered, y = SMD, group = Method, color = Method)) +
  geom_line() +
  geom_point() +
  geom_hline(yintercept = 0.1, color = "black", linewidth = 0.1) +
  coord_flip() +
  theme_bw() +
  theme(legend.key = element_blank()) +  xlab('Variables') +
  labs(title="Standardized Mean Differences for all Models")
```

Question 2

```r
nhefs$qsmk_numeric <- as.numeric(nhefs$qsmk == "Yes")

calculate_effects <- function(data, outcome, formula) {

  # Fit propensity score model
  ps_model <- glm(formula, family = binomial(), data = data)
  data$ps <- predict(ps_model, type = "response")

  # pss
  data$ps_quintile <- cut(data$ps, breaks = quantile(data$ps, probs = seq(0, 1, by = 0.2)),
                          include.lowest = TRUE, labels = FALSE)

  te_quintile <- tapply(data[[outcome]][data$qsmk_numeric == 1],
                        data$ps_quintile[data$qsmk_numeric == 1], mean, na.rm = TRUE) -
                 tapply(data[[outcome]][data$qsmk_numeric == 0],
                        data$ps_quintile[data$qsmk_numeric == 0], mean, na.rm = TRUE)
  PSS <- sum(te_quintile * table(data$ps_quintile) / nrow(data), na.rm = TRUE)

  # IPW
  data$w1 <- data$qsmk_numeric / data$ps
  data$w0 <- (1 - data$qsmk_numeric) / (1 - data$ps)
  IPW1 <- mean(data[[outcome]] * data$w1) - mean(data[[outcome]] * data$w0)
```

```r
IPW2 <- weighted.mean(data[[outcome]], data$w1) - weighted.mean(data[[outcome]], data$w0)

# Perform bootstrap
B <- 1000
nj <- table(nhefs$ps_quintile)
ATE_PSS.boot <- NULL
ATE_IPW.boot <- NULL
ATE_IPW2.boot <- NULL
n <- nrow(nhefs)

for(i in 1:B) {

  data.boot <- data[sample(1:n, n, replace = TRUE), ]
  p1.boot <- glm(formula, data = data.boot, family = "binomial")
  ps.boot <- predict(p1.boot, type = "response")

  ps_quintile.boot <- cut(ps.boot,
      breaks = c(0, quantile(ps.boot, p = c(0.2, 0.4, 0.6, 0.8)), 1), labels = 1:5)

  nj.boot <- table(ps_quintile.boot)

  te_quintile.boot <- tapply(data.boot[[outcome]][data.boot$qsmk_numeric == 1],
      ps_quintile.boot[data.boot$qsmk_numeric == 1], mean) -
  tapply(data.boot[[outcome]][data.boot$qsmk_numeric == 0],
         ps_quintile.boot[data.boot$qsmk_numeric == 0], mean)

  ATE.boot <- sum(te_quintile.boot * nj.boot/ nrow(data.boot), na.rm = TRUE)
  ATE_PSS.boot <- c(ATE_PSS.boot, ATE.boot)

  w1.boot <- data.boot$qsmk_numeric/ps.boot
  w0.boot <- (1-data.boot$qsmk_numeric)/(1-ps.boot)

  ATE_IPW.boot <- c(ATE_IPW.boot,
    mean(data.boot[[outcome]]*w1.boot) - mean(data.boot[[outcome]]*w0.boot))
  ATE_IPW2.boot <- c(ATE_IPW2.boot,
    weighted.mean(data.boot[[outcome]], w1.boot) - weighted.mean(data.boot[[outcome]], w0.boot))

}

results <- data.frame(
  ATE = round(c(PSS, IPW1, IPW2), 3),  # Round ATE values to 3 decimal places
`Standard Error` = round(c(sd(ATE_PSS.boot), sd(ATE_IPW.boot), sd(ATE_IPW2.boot)), 3),
`95% CI` = c(
  sprintf("%.3f, %.3f", PSS - qnorm(0.975) * sd(ATE_PSS.boot), PSS + qnorm(0.975) * sd(ATE_PSS.boot)),
  sprintf("%.3f, %.3f", IPW1 - qnorm(0.975) * sd(ATE_IPW.boot), IPW1 + qnorm(0.975) * sd(ATE_IPW.boot)),
  sprintf("%.3f, %.3f", IPW2 - qnorm(0.975) * sd(ATE_IPW2.boot), IPW2 + qnorm(0.975) * sd(ATE_IPW2.boot))
)
)

rownames(results) <- c("PSS", "IPW1", "IPW2")
colnames(results)[3] <- "95% CI"
colnames(results)[2] <- "Standard Error"
```

```
  return(results)
}

formula_test <- "qsmk ~ sex + race + poly(age, 2) + education + poly(smokeintensity, 2) + poly(smokeyrs
kable(calculate_effects(nhefs, "wt82_71",  formula_test),caption = 'Result for Weight Change')
kable(calculate_effects(nhefs, "sbp",  formula_test),caption = 'Result for SBP')
kable(calculate_effects(nhefs, "dbp",  formula_test),caption = 'Result for DBP')
```